

ИТМО · ТЕСТИРОВАНИЕ ПО · ОСЕНЬ 2026

Основы CI/CD

Модуль 3 · Лекция 6 · В контексте тестирования

Вера Терентьева · Lektor

4 курс

8 октября · Четверг 13:30–15:00

План лекции

01 Зачем CI/CD тестировщику

02 Инструменты CI/CD: GitHub Actions, GitLab CI, Jenkins, TeamCity

03 Пайплайн как код

04 Быстрая обратная связь — ключевой принцип

05 Пирамида как карта скоростей

06 Билд-пайплайн и smoke-тесты

07 Два пайплайна СМП: лёгкий ci.yml и тяжёлый practice-run.yml

08 Makefile, flaky-тесты и отчёты в CI

Зачем тестировщику CI/CD



Тесты без запуска — мёртвый код

Без автозапуска их никто не гоняет — регрессия копится.



CI — машина для пирамиды

Прогоняет unit, API, E2E на каждый коммит.



Быстрая обратная связь = дешёвый дефект

Правило 1:10:100 — ответ за минуты, а не дни.



Не только для девопсов

Условие жизни автоматизации тестирования.

Коммит

Пайплайн

Результат за минуты

Инструменты CI/CD: обзор



GitHub Actions

Встроен в GitHub, YAML в `.github/workflows`. Наш выбор в СМП.



GitLab CI

`.gitlab-ci.yml`, stage-графы с ручными гейтами.



Jenkins

Open-source, тысячи плагинов, свой сервер.



TeamCity

От JetBrains, заточен под JVM, отчёты из коробки.

Как выбирать: где живёт репозиторий (GitHub → Actions, GitLab → GitLab CI), какой стек (JVM → TeamCity) и сколько готовы инвестировать в инфраструктуру (Jenkins — больше всего).

Пайплайн как код

Что это значит

Пайплайн описывается файлом в репозитории, а не настройками в UI

Значит, он версионизируется, ревьюится и воспроизводится

Изменение пайплайна идёт через обычный pull request

Устраняет проблему «настройки живут только в голове админа»

Файлы по инструментам

GitHub Actions — `.github/workflows/ci.yml`

GitLab CI — `.gitlab-ci.yml`

Jenkins — `Jenkinsfile`

Идея одна: декларативно описать, что происходит после коммита

Быстрая обратная связь — ключевой принцип



Pre-commit

секунды



CI

минуты



Nightly

часы

Обратная связь — время от изменения кода до получения результата тестов. Цель: чем короче цикл «изменил → узнал результат», тем лучше.

Разработчик в потоке: 5–10 минут ожидания срывают контекст. Быстрая обратная связь — не бонус, а экономическая необходимость.

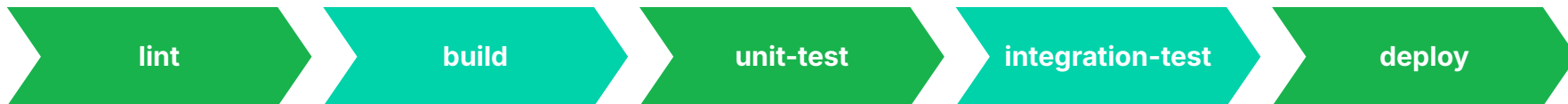
Пирамида как карта скоростей

Уровень	Время прогона	Где запускается
Unit	миллисекунды	pre-commit и CI
Service / API	сотни миллисекунд	CI
E2E	секунды / минуты	nightly или по требованию

Правило: чем ниже уровень теста, тем раньше в пайплайне он запускается. Быстрые unit — обычно белый ящик (покрытие ветвей), медленные API / E2E — чёрный ящик (спецификация и сценарии).

Что такое билд-пайплайн

Билд-пайплайн — последовательность автоматических шагов (стадий) от коммита до результата. Каждая стадия — отдельный набор команд с чётким критерием успеха.



Стадии выполняются **последовательно**, но внутри могут распараллеливаться: в СМП 7 Java-сервисов собираются параллельно через matrix-стратегию.

Упавшая стадия останавливает пайплайн — код не идёт дальше. Это принцип **fail fast**.

Что такое smoke-тесты

Smoke-тест — минимальная проверка, что система «дымит», то есть запустилась и базово работает. Термин из электроники: прибор включают и смотрят, не пошёл ли дым.

Проверяет **критические пути**, а не детальную логику. Быстрый отсев: если smoke красный, гонять полный набор бессмысленно.

Smoke — это **позитивный барьер**: «система базово работает». Логически идёт до полного негативного регресса (правило лекции 5).

Структура `scripts/smoke-test.sh` (7 блоков):

1. Health-check всех 11 сервисов + RabbitMQ UI + Dashboard
2. Генерация 500 тестовых карт (`POST /api/cards/generate`)
3. Одиночная транзакция (`POST /api/transactions`)
 - 3a. Bin Lookup: `GET /api/bin/4000000 → issuerId=ISS001`
 - 3b. Notification Service: `GET /api/notifications → 200`
4. Симулятор терминалов: 50 транзакций, сценарий mixed
5. Поиск транзакций (`GET /api/transactions/search`)
6. Статистика дашборда (`GET /api/dashboard/stats`)
7. Последние транзакции (`GET /api/dashboard/recent`)

Стадии пайплайна: что где запускать

Стадия	Состав проверок	Время	Частота
Pre-commit	линтеры + быстрые unit-тесты	секунды	перед каждым коммитом
CI (по push / PR)	линтеры + сборка + все unit + API-тесты	минуты	на каждый push и pull request
Nightly / по требованию	E2E + нагрузочные + миграции	часы	раз в сутки или вручную

МОДУЛЬ 3 · ЛЕКЦИЯ 6

CI/CD в СМП

CI/CD в СМП: два пайплайна

Лёгкий `ci.yml`

на каждый push

- линтеры
- сборка Java-модулей
- сборка frontend

быстрая обратная связь · без Docker

Тяжёлый `practice-run.yml`

вручную, перед сдачей

- Docker-сборка
- smoke / e2e / load
- полная проверка

`workflow_dispatch` · дорого

Почему два пайплайна, а не один: Docker-сборка 11 образов слишком дорого, чтобы гонять её на каждый коммит.

Разбор ci.yml: лёгкий пайплайн

```
on:
  push:          # любая ветка, без docs/ и *.md
  pull_request:  # в main, только services/**

jobs:
  base-lint:      # pre-commit-хуки
  build-common:   # needs: base-lint
  java-services:  # needs: [base-lint, build-common]
    strategy:
      matrix:      # 7 сервисов параллельно
        service: [gateway, switch, authorization, card-management,
                  terminal-simulator, merchant-acquirer, transaction-logger]
    steps: [checkstyle, install -DskipTests]
  frontend-dashboard: # needs: base-lint
```

Цепочка: `base-lint` → `build-common`
и `frontend`; оба → `java-services`.

Fail fast: упавший джоб блокирует
зависимые.

`-DskipTests` : тесты студенты ещё не
написали.

Разбор practice-run.yml: тяжёлый пайплайн

Триггер: `workflow_dispatch` — ручной запуск с выбором практики `1` = smoke, `5` = e2e, `6` = load-smoke. Исполняется только выбранный джоб.

1. Подготовка

Buildx, кэш Maven/npm через `buildkit-cache-dance`.

2. Подъём стека

`docker compose build` → `up -d` →
ожидание health-check + пауза 15 сек.

3. Прогон и гигиена

smoke-скрипт → логи при падении →
`down -v`.

Одна docker-сборка, один поднятый стек — ровно под ту проверку, которую сдаёшь.

Makefile как локальный интерфейс к пайплайну

Единая точка входа для локальных команд. `test` / `lint` — композиция Maven и npm. Кроссплатформенно: Windows — `copy` + `pwsh`.

```
run:    docker compose up -d
stop:   docker compose down
clean:  docker compose down -v --remove-orphans
build:  docker compose build
smoke:  ./scripts/smoke-test.sh
test:   mvn-test npm-test
lint:   mvn-lint npm-lint
```

Локально = в CI. `make smoke` $\approx$ `smoke-tests`, `make lint/test` $\approx$ `ci.yml`.

Нет расхождения «на моей машине работало».

Flaky-тесты: почему CI перестают доверять

Flaky-тест — на одном коде то падает, то проходит. Падение перестаёт быть сигналом — команда учится игнорировать красный билд.

Причины

- асинхронный старт, тайминги
- гонки данных между тестами
- зависимость от порядка
- нестабильное окружение

Лечение

- тайминги → явные ожидания (Awaitility)
- гонки → изоляция данных
- окружение → карантин

Ретрай — только временная мера, не «зелёный по ретрау».

Отчёты и уведомления в CI



Логи на падении

`docker compose logs --tail=100` — первая помощь.



Уведомления в PR

Статус пайплайна виден прямо в pull request.



Сдача артефакта

Issue + ссылка на зелёный Actions run.



Без отчётов — чёрный ящик

Упало что-то, а что — непонятно.

Интерактив и выводы

1

checkstyle упал в java-services

Что произойдёт с остальными джобами? Как устроена цепочка needs?

2

smoke падает раз в 5 прогонов

Какие гипотезы? Как диагностировать и что лечить первым?

3

Тяжёлый пайплайн идёт 40 минут

Как ускорить без потери покрытия?

Выводы: пайплайн как код · fail fast · flaky лечим изоляцией, ожиданиями и карантином, а не ретраем.

Дальше — модуль 4: Unit-тесты на Java / JUnit 5.

Источники

Humble, J., & Farley, D. (2010). *Continuous Delivery*. Addison-Wesley. — Главы 1–3 (принципы CD), 8 (Automated Testing).

Winters, T., Manshreck, T., & Wright, H. (2020). *Software Engineering at Google*. O'Reilly. — Глава 11 (Testing Overview), Глава 12 (Testing at scale, CI).

Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate*. IT Revolution Press. — Метрики CI/CD, скорость обратной связи.

Spillner, A., Linz, T., & Schaefer, H. (2014). *Software Testing Foundations* (4th ed.). Rocky Nook. — Глава 6, 6.3.1 (стоимость дефекта).

Meszaros, G. (2007). *xUnit Test Patterns*. Addison-Wesley. — Flaky-тесты, изоляция данных, карантин.

GitHub Actions documentation — docs.github.com/en/actions.

TeamCity documentation — jetbrains.com/teamcity/documentation.

The background is a dark blue gradient. It features several glowing green lines that flow across the frame, resembling data paths or circuitry. Interspersed among these lines are stylized, glowing green icons of classical buildings with columns, possibly representing institutions or legacy systems. There are also several glowing green credit cards floating in the scene, some of which appear to be connected to the data lines. The overall aesthetic is futuristic and technological.

Вопросы?